@api_client

Appears in: **DL-02**

shared/api_client.py:1-95

(same shared-client sentence as above)

@api_client

shared/api_client.py:1-95

```python
import json
import os
from pathlib import Path

import httpx

API_BASE = os.getenv("API_BASE_URL", "http://localhost:8000")
MOCK_DIR = Path(__file__).parent.parent / "mock_data"


# ── Offline loaders (from JSON files -- no server needed) ──────────────

def load_inventory() -> list:
    return json.loads((MOCK_DIR / "inventory.json").read_text())


def load_suppliers() -> list:
    return json.loads((MOCK_DIR / "suppliers.json").read_text())


def load_purchase_orders() -> list:
    return json.loads((MOCK_DIR / "purchase_orders.json").read_text())


def load_invoices() -> list:
    return json.loads((MOCK_DIR / "invoices.json").read_text())


# ── Live API calls ─────────────────────────────────────────────────────

def get_inventory() -> list:
    return httpx.get(f"{API_BASE}/api/inventory", timeout=5.0).json()


def get_suppliers() -> list:
    return httpx.get(f"{API_BASE}/api/suppliers", timeout=5.0).json()


def get_purchase_orders() -> list:
    return httpx.get(f"{API_BASE}/api/purchase-orders", timeout=5.0).json()


def get_invoices() -> list:
    return httpx.get(f"{API_BASE}/api/invoices", timeout=5.0).json()


def inject_scenario(name: str) -> dict:
    r = httpx.post(f"{API_BASE}/api/scenario/{name}", timeout=5.0)
    r.raise_for_status()
    return r.json()


def reset_scenario() -> None:
    try:
        httpx.post(f"{API_BASE}/api/scenario/reset", timeout=5.0)
    except Exception:
        pass


def log_decision(trigger: str, analysis: str, decision: str,
                 confidence: float, metadata: dict) -> dict:
    try:
        r = httpx.post(f"{API_BASE}/api/agent-log", json={
            "trigger": trigger,
            "analysis": analysis,
            "decision": decision,
            "confidence": confidence,
            "metadata": metadata,
        }, timeout=5.0)
        return r.json()
    except Exception:
        return {"logged": False}


def _post_purchase_order(payload: dict) -> dict:
    r = httpx.post(f"{API_BASE}/api/purchase-orders", json=payload, timeout=5.0)
    r.raise_for_status()
    return r.json()


def queue_approval(item: dict) -> dict:
    try:
        r = httpx.post(f"{API_BASE}/api/approvals", json=item, timeout=5.0)
        return r.json()
    except Exception:
        return {"queued": False}


def is_server_up() -> bool:
    try:
        httpx.get(f"{API_BASE}/", timeout=2.0)
        return True
    except Exception:
        return False
```

What it shows: The shared HTTP layer (offline JSON loaders + live `/api/...` calls) used by both architectures, so they are interchangeable at run level.

Source: shared/api_client.py:1-95 · image rendered by build_snippets.py